Documentation

GitHub Repository: https://github.com/AloRno/SDMO

Table of Contents

0. A Brief Overview....3
1. Baseline System....4
  1.1. AI-Assisted Development....4
  1.2. Initial Architecture....5
    1.2.1. Communication between components....5
    1.2.2. An Overview on the Cryptography mechanisms....6
    1.2.3. The Lack of features....6
    1.2.4. The Sensor....6
    1.2.5. The Edge Gateway....7
    1.2.6. The Cloud Service....8
    1.2.7. Sequence Diagram of the Baseline System....9
  1.3. Baseline Test, Measurements and initial CI/CD Pipeline....10
    1.3.1. Baseline Tests....10
      test_cryptography_unit....10
      test_handshake_math_correctness....11
      test_end_to_end_integration....11
      test_edge_cases_and_errors....12
        Empty Buffer during data retrieval....12
        Null Inputs during Processing....12
        Corrupted Data or Invalid Padding....12
    1.3.2. Performances of the Baseline System....12
    1.3.3. Coverage of the Baseline Tests....13
    1.3.4. CI/CD Pipeline....13
2. Testing, Reviewing, Correcting and Improving the Baseline System....14
  2.1. Data Corruption....14
    2.1.1 Statistical Solution....14
    2.1.2. More Integrity Guarantees Required....15
    2.1.3. Technical Debt....15
  2.2. Message Persistence in Cloud Service (SQLite)....16
    2.2.1. Testing....16
    2.2.2. Fixing the redundancy....16
  2.3. ML-KEM Implementation....17
    2.3.1. Testing....17
    2.3.2. Performance Comparison....18
    2.3.3. Functionalities Comparison....19
    2.3.4. Technical Debt....20
  2.4. Implementing a “demo”....21
    2.4.1. Rich (https://github.com/textualize/rich)....21
    2.4.2. Plotext (https://pypi.org/project/plotext/)....21
    2.4.3. Testing the TUI....21
    2.4.4. Technical Debt....21

The monolithic architecture of *main.py*....22
Lack of thread-safety....22
2.4.5. Resolving The Technical Debt....22
2.5. The CI/CD Pipeline....24
2.5.1. The Monitoring System....24
2.5.2. Building the Deployment Environment....25
2.5.3. Health checks....25
2.5.4. Fixing Docker Volumes....26
2.5.5. Resolving Telemetry Monitoring Stalls in the CI/CD Pipeline....26
3. About the use of AI....28
3.1. AI Usage for the integration of ML-KEM....28
3.2. AI Usage for Demo creation and TUI rejection....29
3.3. AI Usage for building and improving the CI/CD Pipeline....30
4. Remaining Technical Debts....31
4.1. Lack of Authentication....31
4.2. Weak KDF (Key Derivation Function)....31
4.3. Hard-coded parameters....31

0. A Brief Overview

The implemented system simulates an end-to-end telemetry monitoring pipeline comprising three components: a Legacy Sensor, an Edge Gateway, and a Cloud Service.

The system continuously samples temperature readings at the edge, transmits them across a segmented security perimeter, and processes them within a cloud backend.

The primary objective of this document is to evaluate the modernization of legacy cryptography—specifically transitioning from classical Diffie-Hellman to quantum-resistant ML-KEM—developed and maintained with the assistance of LLMs.

Rather than introducing the overhead of a distributed physical network stack, the architecture is realized as an in-process simulated model. The system components run concurrently across coordinated threads communicating via in-memory FIFO message buffers.

The pipeline is packaged into a multi-container environment configured via Docker Compose, integrating live telemetry metrics, supported by automated GitHub Actions CI/CD workflows and container health checks.

1. Baseline System

In this first part we are going to explain how we used the AI to assist us in the development phase of the legacy baseline system. We will focus on relevant prompts, outputs and decisions.

Then we will proceed with the analysis of the baseline system, the initial architecture and features.

1.1. AI-Assisted Development

The baseline system was developed using Google Gemini as underlying model through an incremental approach. Rather than generating the entire system in a single unconstrained prompt, the architecture was decomposed into its three core components based on the initial UML specification that you can see in further section. The three components are:

- The Legacy Sensor
- The Edge Gateway
- The Cloud Service

To avoid complexity of low-level socket programming and keep focus on cryptographic modernization, the AI and developer agreed on an in-memory simulation using FIFO buffer lists across thread boundaries.

Full raw prompt transcripts and conversational logs are preserved in `IA_Conversation/` folder. This folder has been managed entirely by an AI agent that was responsible to record the conversations across the different phases of the project. In order to keep this part as concise as possible and to avoid just copy-paste of the full conversation we will summarize the important parts.

First Interaction – Understanding the Architecture and designing the Baseline System

We used the LLM to help us define the baseline architecture of the system, the roles of the various components and to draft some UML diagrams to rely on during the implementation of the baseline system. This was more like a brainstorming phase to set everything up.

Details can be seen at:

`IA Conversation/Phase0_Initial_Architecture_Design/Architecture_Definition.md`

Second Interaction – Implementing the Baseline System

During this phase we instead focused on the implementation of the baseline components. We asked the LLMs to write small portions of code with each prompt in order to be able to easily verify it. We instructed the LLM in order to follow the UML diagrams previously generated in order to have a system that is coherent with the design we came up thanks to the previous interaction. In this interaction we also instructed the model to avoid complex features like TLS and real network connection in order to keep the whole system as simple as possible to understand. We also asked the

LLMs to implement a DH key-exchange instead of an RSA key encapsulation mechanism because it felt more “natural” to us as mechanism to agree on a shared secret.

Details can be seen at:

`IA Conversation/Phase0_Initial_Architecture_Design/Baseline_Implementation.md`

The whole baseline system was then designed to be functional and easy understandable, every time the LLMs suggested complex feature we relied on something that instead simulates that feature in order to avoid useless complexity (Exchange of messages via buffers instead of real TCP).

Finally we need to point out a final thing. We changed the type of string returned by the Sensor relative to the one chosen during the design part. Instead of random strings in the form <*string*>|<*timestamp*> we decided to go for a meaningful data type: <*temperature*>|<*timestamp*>.

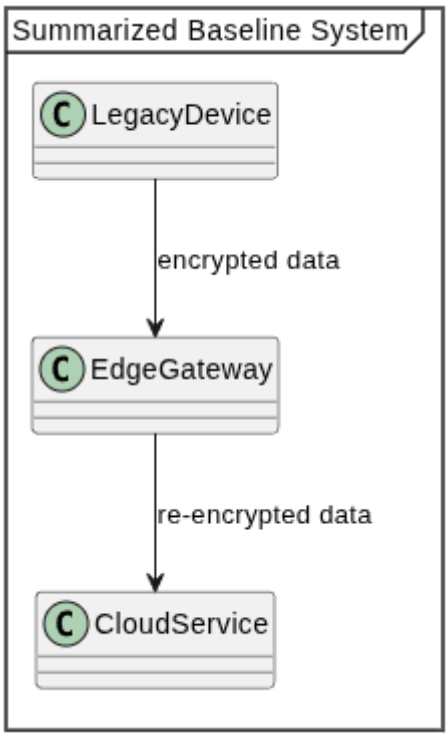
The class diagram that the LLM followed in order to implement the system will be discussed and analyzed in next section.

1.2. Initial Architecture

The Legacy system consist of three main components:

- The Sensor (Represented as “Legacy Device”).
- The Edge Gateway.
- The Cloud Service

1.2.1. Communication between components



In order to keep the system as simple as possible the communication between the devices does not happen via a real TCP connection. In this initial architecture the communication between the devices is implemented via buffers: a sender (e.g. the Edge Gateway) when need to send data to receiver (e.g. the Cloud Service) just put the data into a buffer that follows a FIFO policy (usually implemented as a List object), the receiver is then responsible for retrieving the data from that buffer.

1.2.2. An Overview on the Cryptography mechanisms

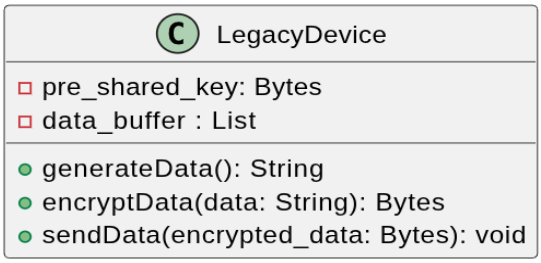
In order to simulate the strict hardware and power constraints typical of legacy IoT sensors, the Sensor have a hard-coded AES symmetric key shared with the Edge Gateway. The Sensor encrypts its data with this static key and transmits it to the Gateway. The Edge Gateway is then responsible for decrypting it and enciphering it again with another key that is established together with the Cloud service thanks to a well-known key-agreement protocol: The Diffie-Hellman protocol. Since this protocol is vulnerable to quantum attacks, ML-KEM will be integrated in future phases of the project.

1.2.3. The Lack of features

Since the first part of the project focuses on modernizing the key agreement mechanism between the Cloud service and the Edge Gateway, extra functionalities such as databases and GUI to visualize the data stored in the Cloud have not been implemented intentionally.

With this system, that aims to mimic a wireless connection, and lacks some advanced functionality, we are both able to start with a simple architecture to understand and leave room for future improvements.

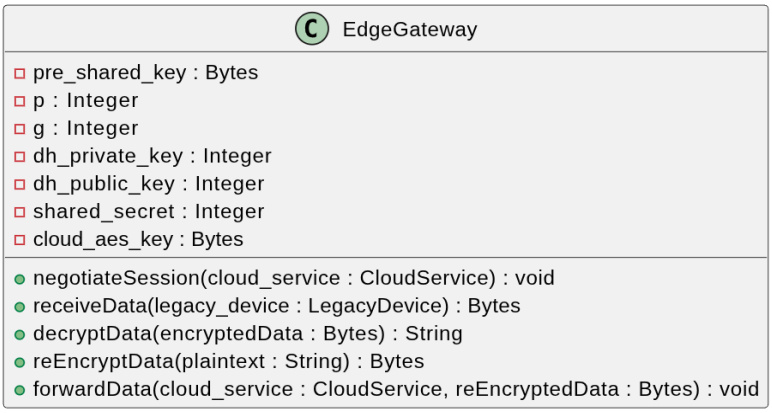
1.2.4. The Sensor



The Sensor (Implemented as `LegacyDevice`) simulates an immutable component of the system that is operating under strict hardware and power constraints. It cannot perform dynamic key exchange and therefore relies on a static pre-shared symmetric key.

Attribute/Method	Type/Signature	Description
`pre_shared_key`	`bytes`	Static AES-128 key shared with the Gateway
`data_buffer`	`list`	FIFO queue holding encryption packets pending transmission.
`generateData()`	`() -> str`	Generates the data (`<temperature>|<timestamp>`), with probabilistic hardware fault injection.
`sendData()`	`(data: bytes) -> None`	Enqueues encrypted payload into `data_buffer`.

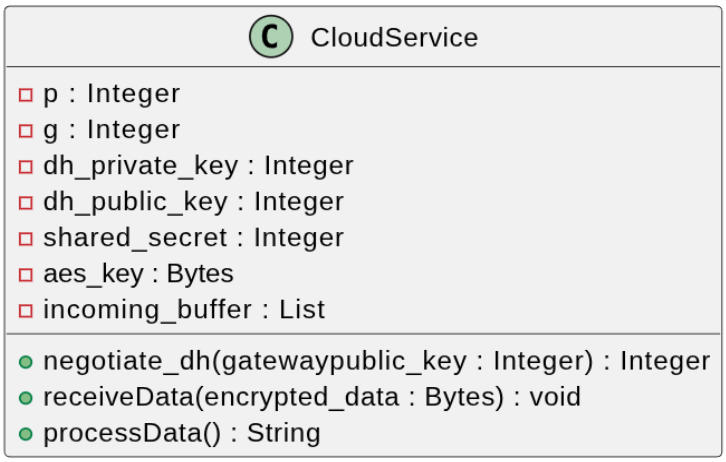
1.2.5. The Edge Gateway



The Edge Gateway serves as the cryptographic bridge connecting the legacy field sensor to the cloud backend. It decrypts legacy sensor data, validates its integrity, and re-encrypts it using a session key negotiated with the Cloud Service.

Attribute/Method	Type/Signature	Description
`pre_shared_key`	`bytes`	Static AES key matching the sensor for inbound payload decryption.
`p, g`	`int`	Modulo and base parameters for DH key exchange.
`dh_private_key, dh_public_key`	`int`	Ephemeral DH keypair generated per session.
`shared_secret` / `cloud_aes_key`	`Int / bytes`	Modular exponentiation result and derived AES-256 session key.
`negotiateSession(cloud)`	`(CloudService) -> None`	Executes the DH handshake with the Cloud service.
`receiveData(legacy_device)`	`(LegacyDevice) -> bytes`	Pops encrypted payload from the sensor buffer.
`decryptData(encryptedData)`	`(bytes) -> str`	Decrypts sensor payload using `pre_shared_key.`
`reEncryptData(plaintext)`	`(str) -> bytes`	Re-encrypts validated telemetry under `cloud_aes_key.`
`forwardData(cloud, reEncryptedData)`	`(CloudService,bytes)-> None`	Enqueues re-encrypted payload into the cloud incoming buffer.

1.2.6. The Cloud Service

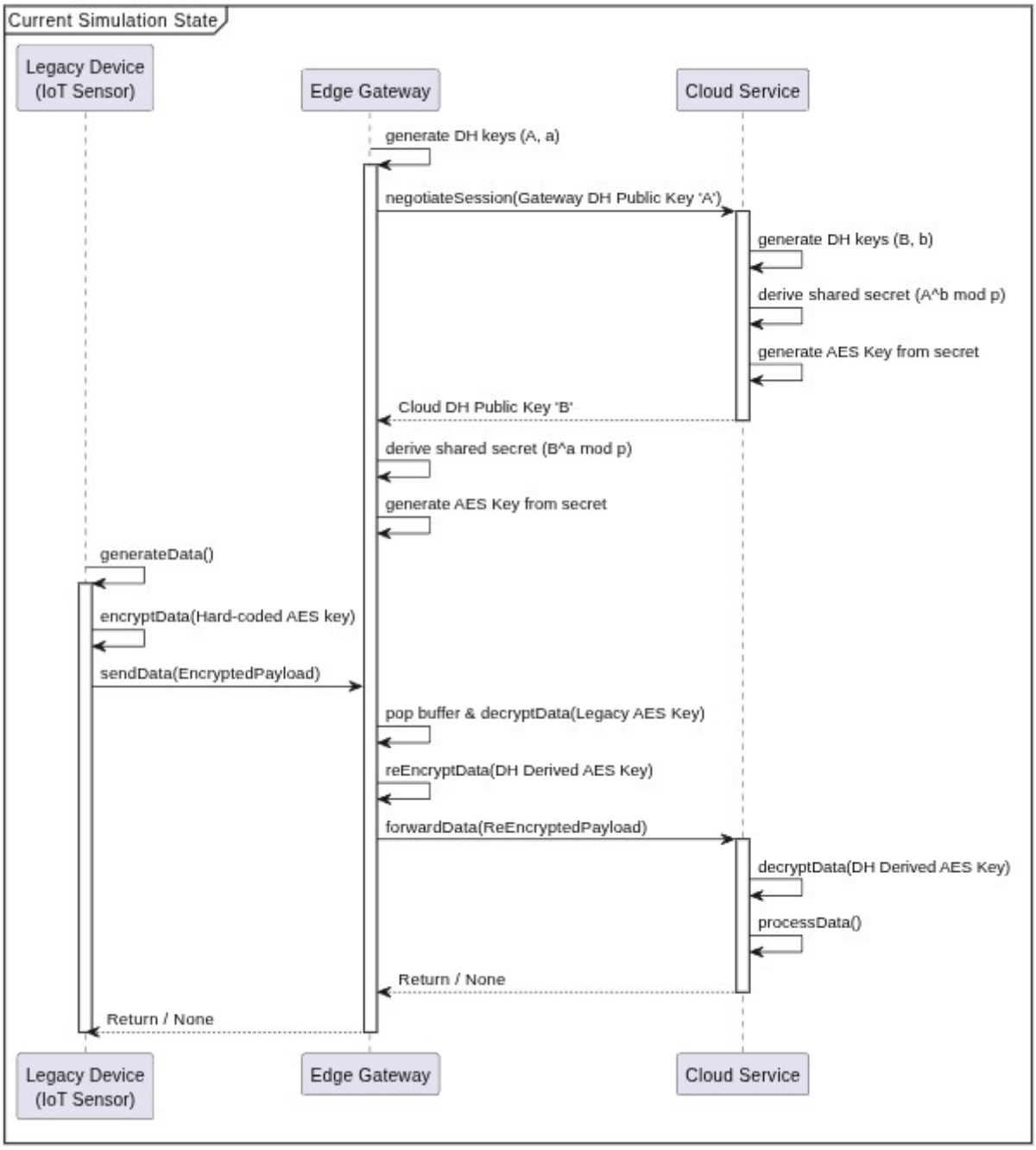


The Cloud Service represents the backend infrastructure terminating the secure connection, validating payload integrity, and processing telemetry.

Attribute/Method	Type/Signature	Description
`p, g`	`int`	DH public domain parameters matching the Gateway.
`dh_private_key, dh_public_key`	`int`	Ephemeral DH keypair generated for session agreement.
`shared_secret` / `aes_key`	`int` / `bytes`	Negotiated shared secret and derived symmetric session key.
`incoming_buffer`	`list`	FIFO queue holding encrypted packets forwarded by the Gateway.
`negotiate_dh(gateway_public_key)`	`(int) -> int`	Completes DH handshake and returns cloud public key.
`receiveData(encrypted_data)`	`(bytes) -> None`	Appends forwarded ciphertext to `incoming_buffer`.
`processData()`	`() -> str`	Decrypts and validates inbound telemetry from `incoming_buffer`.

1.2.7. Sequence Diagram of the Baseline System

The following diagram shows how the system behaves when a data entry is generated from the sensor. This simple communication involves a single message that travels from the Sensor to the Cloud Service.



1.3. Baseline Test, Measurements and initial CI/CD Pipeline

Before introducing the post quantum cryptography key exchange and any other feature in the system, it is necessary to ensure that the legacy system functions correctly. Moreover it is necessary to provide some metrics in order to compare the impact of the future ML-KEM integration with respect to the current key-exchange mechanism.

Then we will proceed with the following test strategy concerning the baseline architecture.

This section aims to document the initial test suites and the first implementation of our CI pipeline.

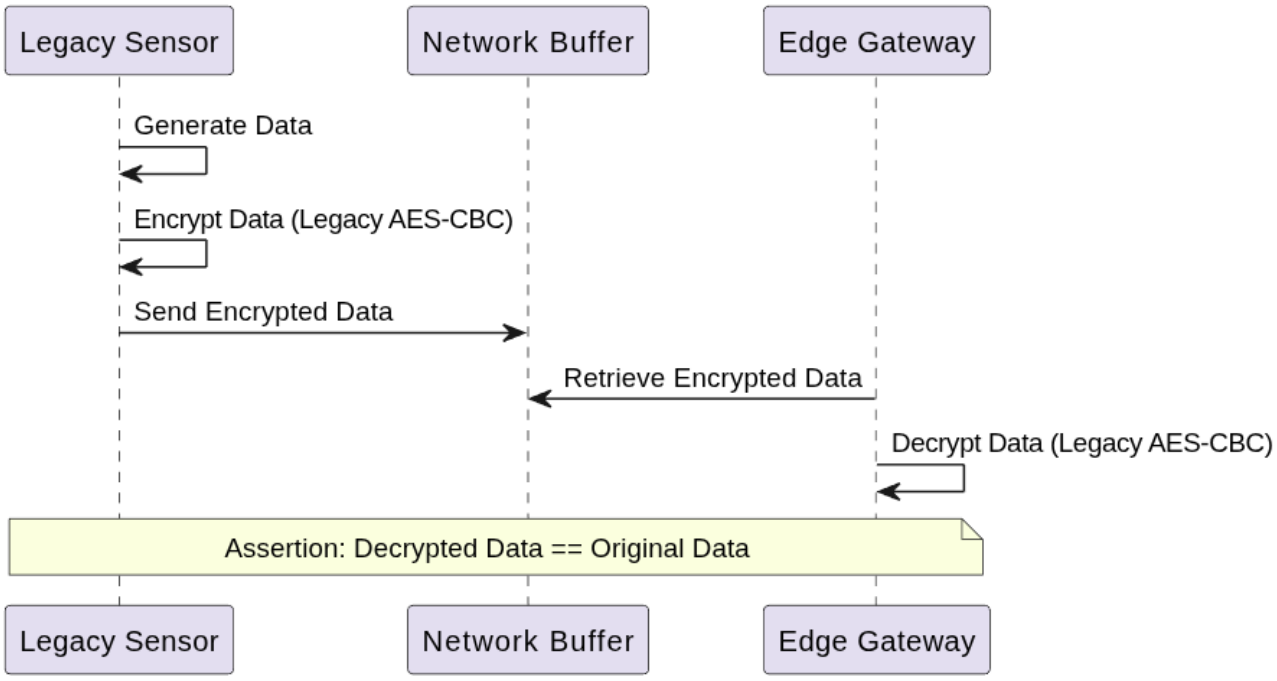
1.3.1. Baseline Tests

The baseline test suite is contained in *tests/test_baseline.py* and it aims to validate the core functionalities, cryptographic correctness and end-to-end integration of the system components. Moreover it sets also a performance baseline for the system that will be compared with analogous metrics at the end of the work.

Now we proceed to show and explain the test cases present in our suite.

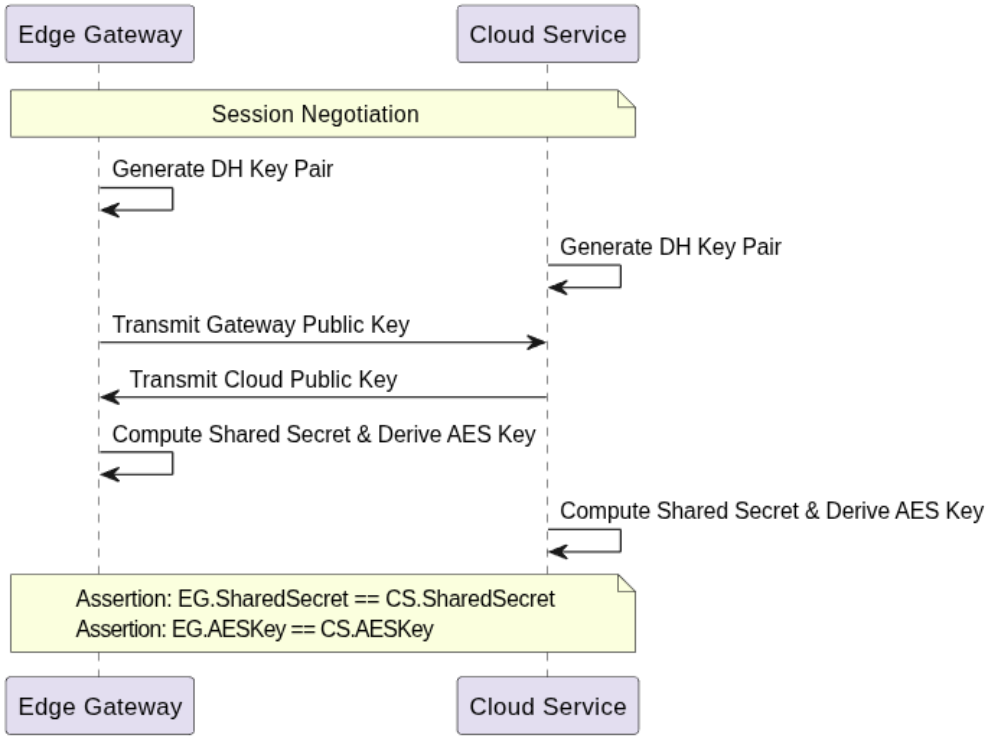
test_cryptography_unit

The purpose of this test is to verify that the data encrypted by the Legacy Sensor using its hard-coded AES key can be successfully and deterministically decrypted by the Edge Gateway. The test works as follow:



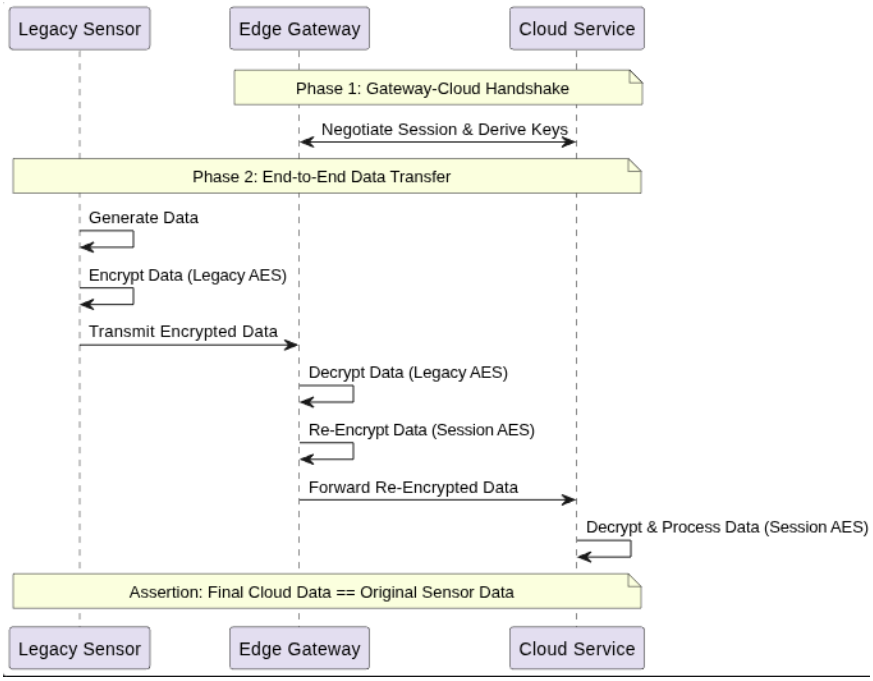
test_handshake_math_correctness

The purpose of this test is to verify the mathematical correctness of the Diffie-Hellman handshake between the Edge Gateway and the Cloud Service. The test works as follow:



test_end_to_end_integration

The purpose of this test is to verify an entire operational flow to ensure that the data correctly goes from one end of the system to the other. Moreover this test is also used to extract metrics about the system. The test work as follow:



test_edge_cases_and_errors

The purpose of this test is to verify the behavior of the system when edge cases occur and how eventual errors are handled. The exact edge cases validated by the test are the followings:

Empty Buffer during data retrieval

If the Edge Gateway tries to fetch data from the receiving buffer when there are no data, it is expected that the function receiveData() return Node.

Similarly if the Cloud Service attempt to retrieve a data from its buffer when it is empty it ie expected that the function returns None.

Null Inputs during Processing

If null inputs are passed to the decryption and re-encryption primitive of the Edge Gateway it is expected that the component is capable of handling it and returns None for this type of invalid inputs.

Corrupted Data or Invalid Padding

In this part of the test we make the Device generate 32 Byte of garbage data, simulating a scenario where data is corrupted, and verify that the Cloud Service is actually capable of catching the failure and return None as expected when trying to decipher it.

1.3.2. Performances of the Baseline System

To critically evaluate the improvement in terms of metrics derived by the introduction of ML-KEM we will first of all measure the performances in terms of memory and time required by the Diffie-Hellman protocol. The metrics we plan to keep track of, in order to compare the two implementations, are:

- Handshake Latency (ms)
- End-to-End Latency (ms)
- Public Key Size (Bytes)
- Ciphertext Size (Bytes)

The test suite documented above has been designed in order to produce metrics relative to the system and the coverage, they can also be retrieved as outputs of the CI pipeline that we will described further on.

In the following table we show the metrics recorded for what concern the baseline system:

Metric	Value
Handshake Latency (ms)	73.3503
End-to-End Latency (ms)	0.1529
Gateway Public Key Size (Bytes)	256
Cloud Public Key Size (Bytes)	256
Cipher text Size (Bytes)	48
Re-encrypted Cipher text Size (Bytes)	48

1.3.3. Coverage of the Baseline Tests

Module	Executable Statements	Missed statements	Coverage
src/cloud_service.py	39	0	100%
src/edge_gateway.py	54	0	100%
src/legacy_device.py	28	0	100%
TOTAL	121	0	100%

1.3.4. CI/CD Pipeline

The CI pipeline leverages GitHub Actions to automatically run tests and report coverage. In this initial implementation the triggers are:

- push on the main branch.
- Pull requests targeting the main branch.
- Workflow dispatch that allows the pipeline to be triggered manually from the GitHub web interface in order to test the configuration.

The pipeline runs the test suite and captures coverage and performance metrics. A mock CD phase has been put in place, in future work a proper one will be implemented.

2. Testing, Reviewing, Correcting and Improving the Baseline System.

In this part we will report the process of testing, reviewing, correcting and improving the baseline system.

This part will involve also the integration of ML-KEM into the system.

2.1. Data Corruption

We noticed that the baseline system does not perform any kind of data integrity validation through the entire pipeline from the legacy sensor to the cloud service. This means that the Cloud could end up receiving corrupted data (think about a sensor malfunction, a transmission problem or someone intentionally tampering with the transmitted data).

Then, in order to avoid filling the Cloud with unreliable data we need to solve the issue. The adopted strategy has been the following:

- Expand the functionalities of the Edge Gateway in order to drop the “bad packets”.
- Expand the functionalities of the Edge Gateway such that it is able to keep a log of the discarded packets.
- Expand the functionalities of the Cloud Service such that it can raise an alarm of some kind if it does not receive updates from the sensor for a long time.

After implementing these changes we updated the test suite accordingly. test_baseline.py has been updated in only to test only the “happy paths”, i.e. all the cases in which the packets are properly composed. It has been created a new test suite: test_failure_handling.py that instead only verifies the handling of corrupted packets.

2.1.1 Statistical Solution

While running the test suite we discovered that some malformed packets are still able to slip through. This happen because the legacy system relies on AES-CBC that does not provide any integrity guarantees. Since the Sensor is a legacy system and we cannot apply fixes here, we must surrender to the idea that fixing this issue is impossible. However we can mitigate it as follow:

First of all we can make the validation performed by the gateway more strict in this way: instead checking only that the temperature is a float as the current implementation does we can enforce a regex. Another measure to reduce the probability that malformed records go through is to verify that the timestamp falls in a certain interval of time. These measures won't catch corruption where 17.5 flips to 16.5 but will drastically reduce the number of undetected corruptions. Then since have a 100% success rate is not feasible we must set a threshold for our tests. We say that we are satisfied (our test passes) if the system catches 80% of the malformed packets.

While conducting further tests we also notice that the probability of a message of passing through depends also on the admitted time interval of the Edge Gateway, tighter is the interval, more probable is to spot corrupted package, if the interval is too tight good packets may be discarded.

Finally to ensure the threshold is respected we made the test suite run about 100 times to ensure as much statistical coverage as possible.

2.1.2. More Integrity Guarantees Required

Since now we have talked only about the communication between the Sensor and the Gateway, now it is time to take into account also the path between the Gateway and the Cloud Service.

As first implementation to enforce integrity guarantees between the Gateway and the Cloud we used CRC32 checksum. The problem is that this approach ensure integrity only against accidental data corruption (e.g. network errors and interferences) and then it is not a good method to enforce integrity when for what concern intentional data tampering.

For this reason the AES-CBC mechanism previously used has been substituted with an authenticated mode of encryption: AES-GCM in order to ensure both confidentiality and integrity.

2.1.3. Technical Debt

Since the Communication between The sensor and the Gateway still relies on AES-CBC in this part of the system the information may be subject to tampering or networks error, this represent an integirity problem, that cannot be solved unless the Sensor itself is replaced.

Details can be seen in:

- *patches/AES_GCM_between_edge_gateway_and_cloud.patch*
- *patches/data_corruption_handling_sensor_to_Gateway.patch*

2.2. Message Persistence in Cloud Service (SQLite)

The Cloud Service in the baseline system do not act like a real cloud, in the sense that currently it lacks the core feature of a cloud service that is the persistent storage of the information.

Sensor telemetry must be reliably stored in a database for historical queries, analytics, and audits.

To solve this issue, the adopted strategy has been the following:

- Introduce persistent storage into the CloudService component using Python's built-in sqlite3 library (requiring zero external dependencies to keep the architecture simple).
- Automatically generate a database table (messages) upon service initialization to store the temperature, timestamp, and original raw message.
- Ensure that the save_message() function is only triggered *after* successful AES-GCM decryption and strict semantic validation, guaranteeing that only healthy data is ever stored.

2.2.1. Testing

After implementing the persistent storage, we expanded our testing coverage. A dedicated automated test (test_cloud_database_persistence) was added to verify the correct insertion of records in the database. This test verifies the correct creation of the table schema and data types. More importantly, it asserts that the Cloud Service completely rejects malformed payloads and refuses to execute the SQL insertion, proving that our database is entirely immune to poisoning from transit corruption or sensor malfunctions. It also verified the end-to-end integration: simulating a legacy sensor reading that successfully traverses the Edge Gateway, gets verified by the Cloud, and finally rests securely in the SQLite database.

2.2.2. Fixing the redundancy

During a subsequent static code review, we identified significant operational redundancy within the SQLite integration. Specifically, the data parsing and schema generation logic were being duplicated on every transaction. We refactored this segment to streamline the data pipeline, eliminating the redundant operations and keeping the entire code base cleaner.

Details can be seen in:

- *patches/sqlite_message_persistence_cloud_service.patch*
- *patches/db_redundancy.patch*

2.3. ML-KEM Implementation

We integrate PQC measure by adopting an hybrid scheme with the DH key exchange already in place. In this way we followed the best-practice standard for the implementation of modern PQC.

The ML-KEM implementation implied several changes and addition to out baseline system that we are going to document here:

First of all we needed to find a library that implements ML-KM, this approach is the best in terms of security since try to write a cryptographic library from scratch is extremely error-prone and would expose the system to severe vulnerabilities, the we relied on the *mlkem Python package* that provided us the official FIPS 203 ML-KEM specification.

As second step we modernized the Cloud Service (*cloudservice.py*) to handle the new hybrid protocol. We had to change the signature of the legacy method *negotiate_dh()* to *negotiate_session()* in order to avoid ambiguities and accommodate the new parameters (the ML-KEM encapsulation key). The method now:

- Compute the DH shared secret.
- Performs ML-KEM encapsulation to generate the ML-KEM shared secret an chipertext.
- Derives the final AES-GCM key by concatenating the DH shared secret and the ML-KEM shared secret via SHA-256.

The third step of this phase was the modernization of the Ege Gateway (*edge_gateway.py*). We modified the method *negotiateSession* to initiate the hybrid protocol that now:

- Generates the ML-KEM encapsulation/decapsulation keys.
- Generates the DH keys.
- Send the keys to the Cloud Service.
- Performs decapsulation upon receiving the ML-KEM cipher text.
- Mixes the retrived ML-KEM secret with the classic DH secret to derive the identical AES-GCM key.

2.3.1. Testing

The previous tests passed flawlessly, then the regression test was successful, we introduced another test suite (*test_mlkem.py*) just to test the basic functionality of the adopted library (it does not invoke any method of the system).

We needed to update the baseline test suite. It was considering only the DH key exchange in the measuring of the metrics, from now on it consider also the metrics of ML-KEM.

2.3.2. Performance Comparison

These are the metrics from the baseline System

Metric	Value
Handshake Latency (ms)	73.3503
End-to-End Latency (ms)	0.1529
Gateway Public Key Size (Bytes)	256
Cloud Public Key Size (Bytes)	256
Cipher Text Size (Bytes)	48
Re-Encrypted Cipher Text Size (Bytes)	48

These are the same metrics with the introduction of the Hybrid Scheme (ML-KEM + DH):

Metric	Value
Handshake Latency (ms)	75.8205
End-to-End Latency (ms)	8.1892
Gateway DH Public Key Size (Bytes)	256
Gateway ML-KEM EK Size (Bytes)	1184
Gateway Total Payload Size (Bytes)	1440
Cloud DH PK Size (Bytes)	256
Cloud ML-KEM CT Size (Bytes)	1088
Cloud Total Payload Size (Bytes)	1344
Ciphertext Size (Bytes)	32
Re-Encrypted Ciphertext Size (Bytes)	43

→ EK stands for (Encapsulation key), it is the conceptual “Public key” of the ML-KEM key agreement. It is the key sent to Cloud.

→ CT The encrypted packet that the Cloud send to the Gateway once it received the EK.

→ The Gateway’s payload consists of both its DH Public Key + the ML-KEM EK. The Cloud’s payload consists of its DH Public Key + the ML-KEM CT.

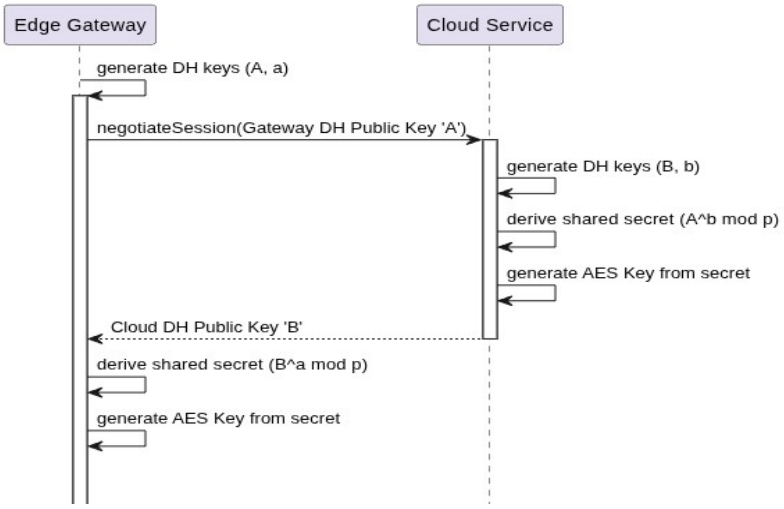
→ Ciphertext Size is the size of the packets that travel between the sensor and the Edge Gateway.

→ Re-Encrypted Ciphertext is the size of packets that travel between the Edge Gateway and the Cloud Service.

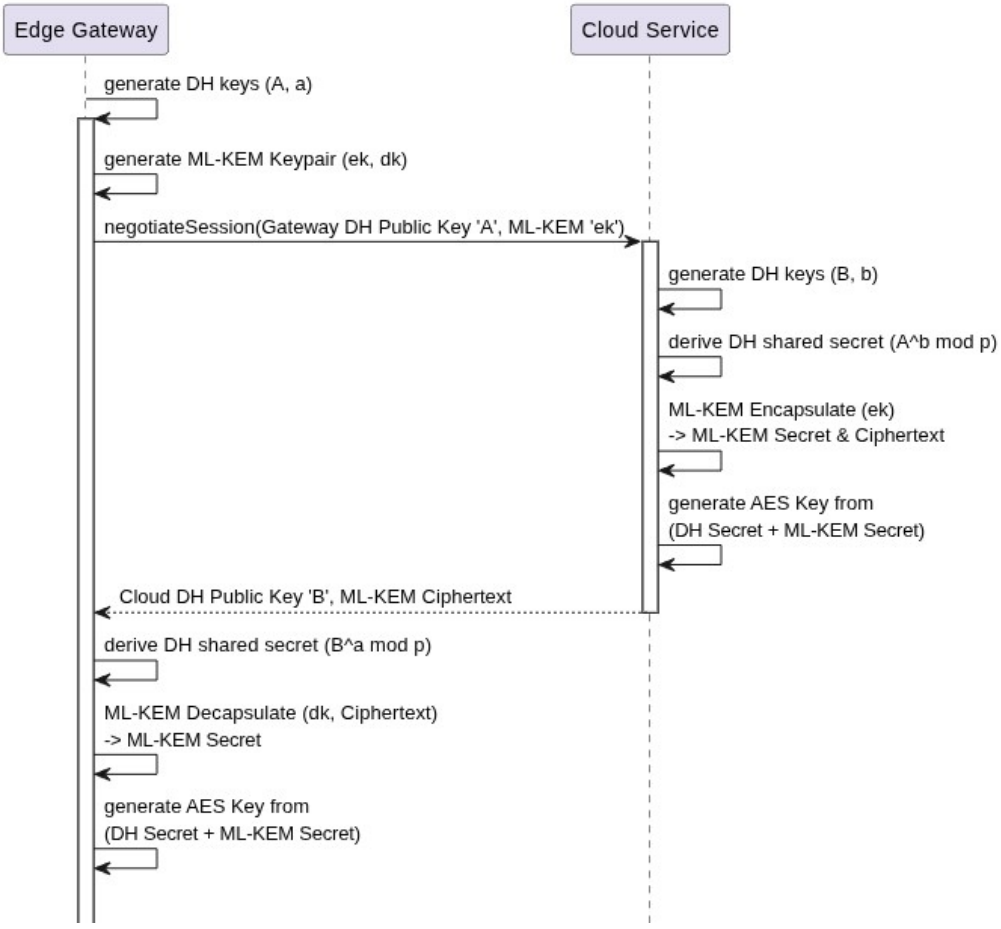
2.3.3. Functionalities Comparison

Here we just see a comparison on “how the system works” before and after the introduction of the Hybrid scheme:

Before: same diagram of the document *Baseline analysis – Initial architecture*



After: Corresponding execution flow after the introduction of the Hybrid scheme:



Details can be seen in:

- *patches/ML_KEM_integration.patch*

2.3.4. Technical Debt

The entire key exchange remain non-authenticated, the system lacks any form of Public Key Infrastructure and digital signatures, then it is vulnerable to Man-In-The-Middle attacks in both directions: Gateway to Cloud and Cloud to Gateway. Then an attacker that gets the public parameters is able to impersonate both ends of the communication.

2.4. Implementing a “demo”

In order to showcase the functionality of the system in real-time, we implemented a multi-threaded architecture with a live dashboard.

The demo utilizes three background threads running concurrently.

- The first thread runs the legacy Sensor, which continuously generates and buffers telemetry data.
- The second thread runs the Edge Gateway, which is responsible for fetching the data from the Sensor's buffer and securely forwarding it.
- The third thread runs the Cloud Service, which decrypts the incoming payloads and updates global statistical metrics.

Finally, the main thread manages a TUI (Terminal User Interface). The TUI visualizes the pipeline's operation by displaying a live temperature graph, a log of incoming data, computed statistics, and a visual alarm that triggers if the sensor drops offline.

In order to construct the TUI we added the following new dependencies to the code:

2.4.1. Rich (**https://github.com/textualize/rich**)

It is used to manage the layouts, panels and the updating of the terminal screen. It is a library used to format the terminal, makes easy to add color, bars and tables.

2.4.2. Plotext (https://pypi.org/project/plotext/)

It is used to generate the graph. It is a library that allows for scatter, line, bar, histograms and date-time plots.

2.4.3. Testing the TUI

While letting run the GUI we noticed that, when simulating on purpose a system failure (setting corruption_probability or failure_probability to 1) the TUI stayed simply quiet without reporting anything. This is not the intended behavior because while developing previous part of the system (the Cloud Service) we designed the system to signal if something was broken.

The problem was due to the fact that the thread running the cloud was not considering the function *sensor_health()* to check the state of the connection.

2.4.4. Technical Debt

We have identified several possible issue in introducing this demo:

The monolithic architecture of main.py

Since all of this has the only purpose to showcase the functionality of the system we overlooked the separation of concern among different components. This resulted in having the *main.py* artifacts that does a lot of stuff: it has logic necessary to the key negotiation, logic to manage the thread and moreover it contains logic related to the TUI.

This is a well-known anti-pattern. Already in this state the artifact is difficult to read and maintain, if more functionalities are added to the TUI without addressing this problem first the codebase will end-up having an unreadable bottleneck.

It is a best practice that the UI logic is keept separated from the rest of the code.

Lack of thread-safety

The components of the system communicate with the TUI via global variables and there is no lock mechanism on these data structures. Then multiple threads have concurrent access to the same data structure. Since our system is pretty simple we do experience race conditions but in a more complex system this could result in a problem. For instance, if the same TUI needs to take into account data from different sensors the chronological order of the data may not be respected at all. To solve this a lock mechanism should be introduced.

This problem in theory is also present for what concern the exchange of packets between the three components of the system, but in this case since the system is properly synchronized via hard-coded time intervals (another very bad practice), the problem is no present.

Details can be seen in:

- patches/TUI_Implementation.patch

2.4.5. Resolving The Technical Debt

While proceeding with the works, during the enhancing of the CI/CD pipeline (see the next section of this document for details). We made the decision to completely remove the TUI from the codebase . While it was useful as brief demo for visualizing the execution flow of the system, it introduces too much complexity to be efficiently handled and maintained, moreover, most of the job of the TUI is done by the monitoring system we implemented (see the next section of this documents for details). The removed the TUI, and consequentially solved great part of the technical debt in the following way:

- We removed the rich and plotext dependencies, drastically reducing the application's footprint and simplifying the Docker container.
- We stripped out the global data buffers, layout logic, and the UI rendering loop, neutralizing the first thread-safety risks analyzed in the section: “T*he monolithic architecture of main.py”*
- We introduced Python's standard logging module to allow the application to run silently in the background, outputting structured logs to standard output.

Moreover in this way we also end-up with cleaner and readable code.

Details can be seen in:

- patches/removing_the_TUI.patch

2.5. The CI/CD Pipeline

The starting point for enhancing the CI/CD pipeline is the baseline system where we had a mock pipeline that just execute the tests and has a placeholder for the multi-container deployment we aim to generate. While in the previously we already set up an initial docker environment to run the demo of the system here we are going to add the live monitoring system, fix eventual issue with the Docker Compose deployment.

2.5.1. The Monitoring System

For the monitoring system we relied on Prometheus, an open source monitoring and alerting toolkit.

After importing the library in our *main.py* we defined three specific metrics:

sdmo_packets_total

This is a counter that tracks the number of packets that have reached the Cloud Service. In Prometheus this data type can only go up and it is useful for an eventual operation team to measure the throughput of the system.

sdmo_temperature_celsius

This metric tracks the live temperature reading currently being processed by the system, the type of data used here is the Gauge. Differently from the counter this kind of data can go up and down.

sdmo_errors_total

This is another counter that acts as a health indicator, It increments every time the Cloud Service detects a failure (i.e. it does not receive packets for long time). This metric is useful because it can be used to launch automated alerts.

These metrics are exposed on an HTTP server that is opened on the port 8000 running in the background of the main thread. Then the metrics can now be seen on *localhost:8000/metrics*. The output is not intended for humans, we are going to address this problem in the next section.

Details can be seen in:

- *patches/Prometheus_initial_implementation.patch*

2.5.2. Building the Deployment Environment

In order to build the deployment environment we are relying on Docker Compose. We will simply add one more container to our current deployment architecture. Up to now we have worked only with a single container that runs the application. In order to integrate properly the monitoring system we will spin up another container that is responsible for running the Prometheus monitoring server. The two containers will then talk to each other and Prometheus will be then able to show the metrics in an human-readable format useful for actively monitoring the system.

So, first of all, we create a configuration file for Prometheus (*prometheus.yml*) that specify to the Prometheus instance where to look for metrics (the “sdmo app” on the port 8000).

Secondly we create the docker compose file (*docker-compose.yml*) that is the core of our deployment that defines the two services.

With this system now we should have a working system, with a monitoring-system that is actually something human-readable, indeed the Prometheus dashboard for the monitoring can be seen at *localhost:9090*.

Details can be seen in:

- patches/Deployment_Environment.patch

2.5.3. Health checks

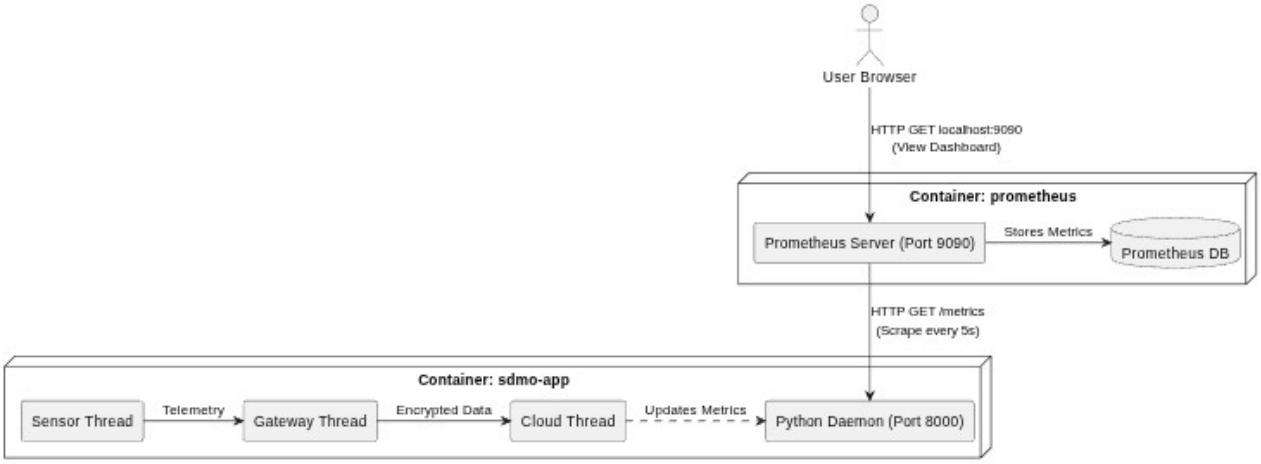
Now comes the moment to improve our CI/CD pipeline with some health checks. Currently the pipeline (*ci.yml*) tries to execute the tests and just have a placeholder for the Docker Compose deployment we implemented in the previous step. However just because our containers are running this does not mean that the application inside it is working. Indeed the tests could pass and the containerized services may also build flawlwssly but what happens if the whole system enters some sort of infinite loop that we did not spot before or remains blocked for other reasons? To solve this we will rely on a “health check” previously configured in out Docker Compose configuration that the CI/CD pipeline will use to know the status of the application.

It works as follows:

Every ten seconds the application tries to fetch the metrics from the Prometheus metrics page, if the Python application is running correctly the Prometheus client will respond with an HTTP 200 OK message. When this message is sent then the application is considered “healthy”. This message exchange is tried three times before the application is considered “unhealthy”, i.e. broken.

Then, with this extra check, the GitHub action will fail not only when the test suite will fail, but also when there are problems in the deployment, effectively protecting the main branch.

The following diagram, show the interaction between the two containers, and how the browser visualize the data from the monitoring system.



Details can be seen in:

- patches/Deployment_Environment.patch

2.5.4. Fixing Docker Volumes

Currently, the SQLite database and Gateway logs generated by the system are stored inside the ephemeral Docker container. Then all the data produced by the application are deleted when the container dies.

To make this a true production-ready architecture, we need to map those directories out to your host machine using Docker Volumes.

In this way the logs of the Edge Gateway will be returned, and in case of eventual transmission failures, these can be used to address eventual problems and the SQLite database build by the Cloud Service is stored on the machine that execute the application, in this way further analysis on the produced data are possible.

Details can be seen in:

- *patches/Docker_Volumes.patch*

2.5.5. Resolving Telemetry Monitoring Stalls in the CI/CD Pipeline

During the final round of system testing, we observed that the Prometheus monitoring dashboard would stop receiving telemetry updated. By directly inspecting the metrics endpoint at *localhost:8000*, we confirmed that packets were still begin successfully generated, indicating problems into the data pipeline rather that a failure of some of the python artifacts. Since we were not able to find the problem on our own, with the assistance of AI, we diagnosed that apparently there were concurrent file-locking collisions within the SQLite database were causing the

application threads to deadlock and freeze the monitoring system. Since this problem completely broke the functionality of the system, we had to find a solution. We proceeded as follow:

- We removed the SQLite database: We completely wiped out the SQLite database from the pipeline. Now the system completely relies on Prometheus' database for historical data storage.
- We reverted the changed performed in the previous intervention (*patch: Docker_Volumes.patch*). In this way we prevented host-level file locks from interfering with the containerized application.

But with these changes we found ourself in the same scenario as before: we are not currently able to read the logs generated by the application because these are confined into the container's file system. But we found out that these can be safely extracted to the host machine via docker commands.

To extract the live Edge Gateway error log to the host's current directory:

```
sudo docker cp sdmo-sdmo-app-1:/app/src/Output_Components/gateway/gateway_error.log ./gateway_error.log
```

Finally, live telemetry data can be accessed using two method:

- The visual way: Navigate to *http://localhost:9090* in the browser to interactively query and visualize data.
- The CLI way: Open a terminal and just fetch the data history for the current session with a command like this:

```
END=$(date +%s)

START=$(($END - 3600))

curl "http://localhost:9090/api/v1/query_range?query=sdmo_temperature_celsius&start=$START&end=$END&step=1s" -o
prometheus_history.json
```

This command is the one used to fetch the historical data about the temperatures.

It is also possible to write automatic scripts to query the HTTP API of Prometheus in order to fetch these data.

Details can be seen in:

- patches/Fixing_Frozen_Pipeline.patch

3. About the use of AI

In this section we are going to dive deeper in the use of AI for what concern the most important activities documented in section 2 (Integration of ML-KEM, implementation of the “demo” and the building and enhancing of the CI/CD pipeline).

3.1. AI Usage for the integration of ML-KEM

The AI was provided with the following key constraints: preserve sensor immutability, enforce pure-Python dependencies, maintain test suite compatibility and adhere to hybrid migration standard combining classical DH with ML-KEM.

The primary prompt was:

```
We are in Phase 2, I am doing the system modernization. The system is ready
to implement ML-KEM. The final scheme must be a Hybrid scheme that combines
DH and ML-KEM following the best-practice of mixing the two secrets into a
single one when PQC is introduced. Update the Edge Gateway and the Cloud
Infrastructure in order to integrate ML-KEM without breaking regression
tests.
```

Follow-up minor prompts requested to maintain the signature of the methods used to establish the key exchange and to update the baseline test suite in order to split DH and ML-KEM metrics into separate rows.

The AI selected the pure-Python *mlkem* package implementing then NIST FIPS 203 (ML-KEM768). Updated the Cloud Service and the Edge Gateway to exchange ephemeral 2048 bit DH keys and ML-KEM encapsulation keys. It also propose upgrading transport encryption to AES-256-GCM. Following developer feedback, it updated the baseline test suite to record the sizes of the exchanged key and ciphertext in the metrics recorded by the tests.

The developer in this phase rejected the AI proposition to update the sensor code to introduce TLS 1.3 due to the fact that the modernization operations should not involve the legacy sensor, since it is the legacy component of the system it can’t be touched.

3.2. AI Usage for Demo creation and TUI rejection

The original plan was to demonstrate the functioning of the pipeline thanks to visual monitoring.

The main prompts used to address this matter have been the followings:

```
I would like to enhance main.py in order to showcase the system pipeline […] A thread that runs legacy_device.py, another that runs edge_gateway.py, and cloud_service.py running in another thread […] I was thinking about a facy TUI separated into two parts, the top showing a graph of teperatures, the bottom divided into live incoming data and statistics (average, maximum, minimum)
```

```
I was looking at main.py, would the GUI work properly if I tweak the failure_probability and corruption_probability? The TUI does not show the alarm message.
```

This part was turbulent, the AI restructured the *main.py* artifact using three background threads coordinated by *time.sleep()* in order to render the dashboard. It introduced new dependencies (*rich* and *plotext* libraries).

The multi thread pipeline structure has been accepted by the developer but the following analysis of the code revealed that the AI omitted the checks for corrupted packets, then the developer modified the code to enforce continuous alarm evaluation.

Further analysis brought the developer to reject the monolithic architecture introduced to implement the TUI, this led to a massive reduction of code complexity and also the dependencies were deleted. The monolithic TUI was replaced using standard logging and Prometheus.

3.3. AI Usage for building and improving the CI/CD Pipeline

The main prompts used to address this matter have been the followings:

```
Create a GitHub Actions CI pipeline in github/workflows/ci.yml that
executes pytest with coverage, saves test metrics to
tests/coverage.txt, and builds the container image
```

```
I was browsing the internet to find a way to implement a monitoring
system for the project, I found out that a standardized way is to use
Prometheus. Could you show me if and how it is possible to integrate
such a tool in this project?
```

The rest of prompts focused on configuring Docker volume mounts for the outputs and resolving CI failures caused by deprecated container commands since the developer was using an outdated version of docker.

The AI generated a multi-staged Dockerfile capable of doing health checks of the system. A *docker-compose.yml* file to orchestrate the main app alongside with a Prometheus container was generated.

The developer accepted the majority of the generated artifacts but had to take action for what concerned the docker volumes by adding host bind mount to guarantee the persistence of the Cloud Database and the gateway logs. The developer also rejected deprecated standalone *docker-compose* binary suggested in early prompts, enforcing modern *docker compose* syntax.

4. Remaining Technical Debts

In this section we are going to document all the technical debts that remain in our system and are not correlated to any of the previous maintenance and operations activities.

4.1. Lack of Authentication

While confidentiality is ensured via ML-KEM768 and AES-256-GCM the system does not implement signatures or certificates, then both the Cloud system and the Edge gateway have no way to prove their identity. Then an attacker can just intercept the connection and perform man in the middle attacks.

4.2. Weak KDF (Key Derivation Function)

The cryptographic key both the Cloud and Gateway rely on is derived by applying SHA256 to the shared secret among the two components of the system. While this yields a perfectly valid AES key, using a standard hash function as a KDF is considered technical debt in modern cryptography because SHA256 lacks context binding.

4.3. Hard-coded parameters

Both the Edge Gateway and Cloud Service come with hard-coded parameters for what concern the DH key exchange, this is another bad practice for what concern cryptography. In order to fix this the initial handshake should be fixed in such a way that the two components of the system agree on a public base for the exponentiation. Moreover also the private exponent that is hard-coded in both the components should be generated at random. Fixing this is effective only if authentication mechanisms (like PKI) are provided, otherwise the system remains vulnerable to Man In the Middle attacks.